Lab 07: Archived Web Pages and the Wayback Machine

INFO 4617/5617 · Week 7 · Chapter 7, Archived Web Pages and the Wayback Machine

Today we code together. Each step below says what we're building and why, and its code cell holds only the comments that frame it: we write the code live, in class. Nothing here is turned in. After each step, a 💬 **Discuss** box holds questions to talk over before the next one; click it to open it.

By the end you'll have read the Wayback Machine's addresses by hand, asked its two APIs which captures of a page exist, fetched the captures as they were saved, and measured how much a document changed from one capture to the next. The case is the chapter's: Facebook's Terms of Service, `facebook.com/terms`, from 2006 to 2024, with stops at Thefacebook in 2004 and CNN in 2000.

Before class: open this notebook in the `webdata` environment (chapter 1), and have your email address ready for the User-Agent. Check that `import gensim` runs; if it doesn't, the chapter's "Document Similarity with Bag-of-Words" gives the one `conda install` line. The Wayback Machine limits how many requests one network address may send, and everyone in this room shares one, so run each cell once, when we do, and expect some cells to wait.

On Friday, run every cell above where we stopped (**Run → Run All Above Selected Cell**).

> **Instructor key.** Every cell is filled in, but **none has run yet**: on 2026-10-21 the Wayback Machine refused this course's cloud machine (429s, and no answer at all from its CDX server). Every code cell is tagged `not-run-live`. Run it from a laptop with `labs/tools/run_key.py`, which runs every cell and removes the tags as cells succeed; then write Step 11's answers from the outputs, and replace this note with the date it ran. The lab follows the textbook at commit d9306b7.

Step 0 — Set up (4 minutes)

Import what we'll use, and define the User-Agent every request will send, in the book's form. The Internet Archive is a nonprofit that serves more than a trillion pages for free, and it is often slow: it answers `429 Too Many Requests` when one address sends too much, and `502` or `503` when it is overloaded. So, as the chapter's "Common Issues to Debug" suggests, send every request through week 5's retrying `Session`, with a longer `backoff_factor`. With `Retry(total=5, backoff_factor=5)`, a failed request is tried again at once, then after 10, 20, 40, and 80 seconds, or after as long as a `Retry-After` header asks. Mount the adapter for `http://` as well as `https://`, because the Availability API (Step 3) hands back `http://` addresses.

```
# Import time, json, os, requests, pandas (as pd), matplotlib.pyplot (as plt), and BeautifulSoup
# Define HEADERS, the book's User-Agent: "WebDataScience/1.0 (your-email@colorado.edu)"
# Make a requests.Session that sends HEADERS, with an HTTPAdapter whose Retry rule retries
#   429, 500, 502, 503, and 504 (total=5, backoff_factor=5); mount it for "https://" and
 ↪  "http://"
import time
import json
import os
import requests
import pandas as pd
import matplotlib.pyplot as plt
from bs4 import BeautifulSoup
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}

session = requests.Session()
session.headers.update(HEADERS)
retries = Retry(total=5, backoff_factor=5, status_forcelist=[429, 500, 502, 503, 504])
```

```
adapter = HTTPAdapter(max_retries=retries)
session.mount("https://", adapter)
session.mount("http://", adapter)
print(session.headers["User-Agent"])
```

💬 Discuss

- In week 5 we gave this retrying session a `backoff_factor` of 1. Why give the Archive five times as long?
- The Archive limits requests per network address, and this room shares one. What should thirty people running the same cell do differently from one person running it at home?
- Week 3 called infrastructure that a community governs "ownership". Who pays for the requests we're about to send, and what do we owe them in return?

Step 1 — Read a capture's address (8 minutes)

Every capture has an address you can read and rewrite, as the chapter's "Moving Through Time" shows:

`https://web.archive.org/web/20040212031928/http://www.thefacebook.com/`

The 14 digits after `/web/` are the time of the capture in UTC, from year to second; everything after them is the original address. Parse the timestamp with `pd.to_datetime()` and a `format`. Then shorten the timestamp to a year: the Wayback Machine redirects you to the capture closest to it, so the address is itself a query. As in week 1, `response.history` keeps the trail of redirects.

```
# Define CAPTURE, the address above. Split off its timestamp and original address
#   (split on "/web/", then on the first "/"), and print the timestamp as a UTC datetime
# Ask the session for "https://web.archive.org/web/2004/thefacebook.com" (a shortened timestamp)
# Print each redirect in response.history (its status and Location header), then the final status
 ↪  and address
CAPTURE = "https://web.archive.org/web/20040212031928/http://www.thefacebook.com/"
stamp, original = CAPTURE.split("/web/")[1].split("/", 1)
print(pd.to_datetime(stamp, format="%Y%m%d%H%M%S", utc=True), original)

response = session.get("https://web.archive.org/web/2004/thefacebook.com", timeout=60)
for step in response.history:
    print(step.status_code, step.headers["Location"])
print(response.status_code, response.url)
```

⏱️ Microsprint (5 minutes, alone or in pairs)

From the chapter's "Try It: Before They Were Famous" box.

Do: pick an address from the box (`google.com` in 1998, `x.com` in 1999) or a site you knew in middle school, and ask for it with only a year as the timestamp. Print where you landed and the page's `<title>`. **Share:** the address, the date of the capture you landed on, and its title.

```
# Microsprint: ask for "https://web.archive.org/web/<year>/<address>" for an address of your
 ↪  choice,
#   then print response.url and the page's <title> (BeautifulSoup(response.text,
 ↪  "html.parser").title)
response = session.get("https://web.archive.org/web/1999/x.com", timeout=60)
print(response.url)
print(BeautifulSoup(response.text, "html.parser").title)
```

💬 Discuss

- `CAPTURE` was made at 03:19 UTC on February 12, 2004. What date and time was that at Harvard, where Thefacebook's users were?
- We asked for 2004 and landed on one capture. If you were studying a page in the week before an event, what could "closest" hide from you?
- Week 1's request to Wikipedia had an empty `response.history`. What is each redirect here telling us?

Step 2 — The archive's wrapper, and `id_` (6 minutes)

What came back in Step 1 isn't the page as it was captured. The Wayback Machine wraps it: it adds its toolbar and scripts, and rewrites the page's links and images to point back into the archive. Open `CAPTURE` in your browser, right-click, and **Inspect** it (week 5) to see the wrapper end at `<!-- END WAYBACK TOOLBAR INSERT -->`; the toolbar's **About this capture** shows when each image on the page was saved. That's convenient for reading and, as the chapter's "Inspecting Archived Pages" puts it, poisonous for measurement. Adding `id_` after the timestamp asks for the original capture, unmodified. Fetch both versions and count what each holds.

```
# Make RAW: CAPTURE with "id_" added after its timestamp
# For CAPTURE and then RAW: fetch it, parse it, and print how many <script> tags it has, how many
 ↪  links
#   (<a> with an href), and how many of those point into the archive ("/web/" in the href); sleep
 ↪  2 seconds
# Print the first three links of RAW
RAW = CAPTURE.replace(stamp, stamp + "id_")
for version in [CAPTURE, RAW]:
    soup = BeautifulSoup(session.get(version, timeout=60).text, "html.parser")
    hrefs = [a["href"] for a in soup.find_all("a", href=True)]
    into_archive = [href for href in hrefs if "/web/" in href]
    print(f"{len(soup.find_all('script'))} scripts, {len(hrefs)} links, "
          f"{len(into_archive)} into the archive: {version}")
    time.sleep(2)
print(hrefs[:3])
```

💬 Discuss

- Suppose you counted `<script>` tags on wrapped captures to study how JavaScript spread after 2004. What would you have found, and would anything have warned you?
- In week 5, the Inspector showed the page after scripts ran and View Source showed what the server sent. Which is `RAW` closer to, and which does your browser show you for a capture?
- Every link in `RAW` carries a `PHPSESSID`. What does that tell you about the crawler, and about what a logged-in student in 2004 would have seen instead?

Step 3 — The Availability API, across three eras (10 minutes)

The Availability API answers one question in JSON: which capture of this address is closest to this date? Pass the address and the date as a `params` dictionary; `requests` encodes them, which matters because one of the values is itself a URL. The chapter's `get_snapshot_stats()` asks the API, fetches the capture it names with `id_`, and counts the capture's links, text, images, and scripts. Run it for CNN's home page in August of 2000, 2012, and 2024, pausing two seconds between dates as the chapter does. The chapter's table of results is illustrative; ours will be real.

We copy the chapter's function with one change, made in every function today: it sends its requests through `session`, with a `timeout`, instead of calling `requests.get()` directly.

```
# Define get_snapshot_stats(url, timestamp, headers=None), as in the chapter: ask the
 ↪  Availability API for the
#   closest capture (return None if "archived_snapshots" is empty), fetch it with "id_" after its
 ↪  timestamp,
```

```
#   and return the year and the counts of <a> tags, text characters, <img> tags, and <script>
 ↪  tags
# Run it for "www.cnn.com" at 20000815, 20120815, and 20240815, sleeping 2 seconds between; show
 ↪  a DataFrame
def get_snapshot_stats(url, timestamp, headers=None):
    """Retrieve a Wayback snapshot and compute basic statistics."""
    try:
        api_response = session.get(
            "https://archive.org/wayback/available",
            params={"url": url, "timestamp": timestamp},
            headers=headers,
            timeout=60,
        )
        api_response.raise_for_status()
        api_data = api_response.json()

        if not api_data.get("archived_snapshots"):
            return None

        # Fetch the original capture (id_), not the toolbar-wrapped version, so the statistics
         ↪  measure the page and only the page
        closest = api_data["archived_snapshots"]["closest"]
        snapshot_url = closest["url"].replace(
            closest["timestamp"], closest["timestamp"] + "id_"
        )
        page_response = session.get(snapshot_url, headers=headers, timeout=60)
        page_response.raise_for_status()
    except requests.exceptions.RequestException as e:
        print(f"Request failed for {url} at {timestamp}: {e}")
        return None

    soup = BeautifulSoup(page_response.text, "html.parser")

    return {
        "year": timestamp[:4],
        "num_links": len(soup.find_all("a")),
        "text_length": len(soup.get_text()),
        "num_images": len(soup.find_all("img")),
        "num_scripts": len(soup.find_all("script")),
    }

stats = []
for ts in ["20000815", "20120815", "20240815"]:
    result = get_snapshot_stats("www.cnn.com", ts, headers=HEADERS)
    if result:
        stats.append(result)
    time.sleep(2)  # Be respectful of the Archive

comparison = pd.DataFrame(stats)
print(comparison)
```

⏱️ Microsprint (5 minutes, alone or in pairs)

A page-statistics version of Additional Exercise 2, which compares two competing organizations.

Do: run `get_snapshot_stats()` for a competitor of CNN's (`www.nytimes.com`, `www.foxnews.com`,

`www.usatoday.com`) in August 2000 and August 2024. **Share:** the site, and its numbers of links and scripts in each year.

```
# Microsprint: run get_snapshot_stats() for another news site at 20000815 and 20240815,
#   sleeping 2 seconds between, and show the rows that came back as a DataFrame
rival = []
for ts in ["20000815", "20240815"]:
    rival.append(get_snapshot_stats("www.nytimes.com", ts, headers=HEADERS))
    time.sleep(2)
print(pd.DataFrame([row for row in rival if row]))
```

💬 Discuss

- `get_snapshot_stats()` reports the year we asked for, not the capture's own timestamp. How far from August 15 could the capture be, and how would you check?
- Which count grew most from 2000 to 2024? Week 6's "When Static Isn't" warned about pages built by scripts: what can `requests` not see on the 2024 page?
- Each call sends two requests with no pause between them, then sleeps 2 seconds. Is that within week 2's default of one request a second?

Step 4 — The CDX Server: every capture of a page (6 minutes)

The Availability API finds one capture; the CDX Server API lists all of them. Ask it about `facebook.com/terms` with the chapter's parameters: `output=json`, the fields (`fl`) we want, `filter=statuscode:200` for successful captures only, and `collapse=digest` to drop each capture whose content hash matches the one before it. We add one parameter the chapter leaves out, `to=20260930`, so that a rerun next year stops at the same date. The first row of the answer holds the column names.

```
# Request the CDX Server (https://web.archive.org/cdx/search/cdx) with params:
 ↪  url="facebook.com/terms", output="json",
#   fl="timestamp,original,statuscode,length", filter="statuscode:200", collapse="digest",
 ↪  to="20260930" (timeout=120)
# Make a DataFrame from rows 1 onward, with row 0 as the columns; convert timestamp (format
 ↪  "%Y%m%d%H%M%S")
#   and length (pd.to_numeric); print the number of captures and the date range
cdx_url = "https://web.archive.org/cdx/search/cdx"
params = {
    "url": "facebook.com/terms",
    "output": "json",
    "fl": "timestamp,original,statuscode,length",
    "filter": "statuscode:200",  # Server-side: only successful captures
    "collapse": "digest",        # Server-side: skip consecutive identical captures
    "to": "20260930",            # Fix the end, so a rerun gives the same answer
}
response = session.get(cdx_url, params=params, timeout=120)
response.raise_for_status()
data = response.json()

df = pd.DataFrame(data[1:], columns=data[0])
df["timestamp"] = pd.to_datetime(df["timestamp"], format="%Y%m%d%H%M%S")
df["length"] = pd.to_numeric(df["length"], errors="coerce")

print(f"Total snapshots: {len(df)}")
print(f"Date range: {df['timestamp'].min()} to {df['timestamp'].max()}")
print(df.head())
```

💬 Discuss

- How many requests did that take? How many would the Availability API need to tell us the same thing?
- `collapse=digest` keeps a capture whenever its bytes differ from the one before. Would a new date in the page's footer, or a new ad, count as a new version?
- Week 4's JSON came as a list of dictionaries. What shape is this JSON, and why did we need `data[0]`?

Step 5 — Plot the archival record (6 minutes)

Two questions for the record: how often was a new version captured, and how big was each one? Set the timestamp as the index and `resample()` by quarter (`"QS"`), counting captures and averaging `length`, as the chapter's "Filtering and Analyzing CDX Data" does. Plot both, one above the other, on a shared time axis. The CDX Server's `length` is the size of the archived record, compressed, not the size of the page in your browser: good for spotting jumps, not for saying how big the page was.

```
# Index df by timestamp and resample by quarter ("QS"): count the captures and average length
# Plot both, one above the other with a shared x axis (plt.subplots(2, 1, sharex=True)),
#   each with a title and a labeled y axis; then print the three busiest quarters
df_ok = df.set_index("timestamp")
quarterly = df_ok.resample("QS").agg(
    captures=("length", "size"),
    mean_size=("length", "mean"),
)

fig, axes = plt.subplots(2, 1, figsize=(10, 6), sharex=True)
quarterly["captures"].plot(ax=axes[0])
axes[0].set_ylabel("Captures per quarter")
axes[0].set_title("facebook.com/terms: distinct captures (status 200, collapsed on digest)")
quarterly["mean_size"].plot(ax=axes[1])
axes[1].set_ylabel("Mean record size (bytes, compressed)")
axes[1].set_title("facebook.com/terms: mean size of the archived record")
axes[1].set_xlabel("Quarter")
plt.tight_layout()
plt.show()

print(quarterly.sort_values("captures").tail(3))
```

💬 Discuss

- With `collapse=digest`, every row is a change in bytes. Is the busiest quarter the quarter Facebook changed its terms most, or the quarter the Archive paid most attention?
- Pick a sudden jump in size. What would you open to tell a redesign from a move behind a login?
- In week 1 we plotted an article's daily views. Whose behavior does this plot measure: Facebook's, its readers', or the Archive's?

Step 6 — What the calendar's colors mean (9 minutes)

The Wayback calendar colors each day by the HTTP status code (week 5) the crawler received: blue for 2xx, green for 3xx, orange for 4xx, red for 5xx. The CDX Server reports the same codes in `statuscode`, so drop the filter and look at what Step 4 left out. Ask for every capture of `facebook.com` from 2005, the year the address changed hands, and cross-tabulate month by status code with `pd.crosstab()`. The chapter's "What the Colors Mean" says what to expect from April to August.

```
# Ask the CDX Server for url="facebook.com", output="json", fl="timestamp,statuscode",
#   from "2005" to "2005", with no filter and no collapse
# Make a DataFrame, add a month column (the timestamp's first 6 characters), and print
 ↪  pd.crosstab() of month by statuscode
```

```
params = {"url": "facebook.com", "output": "json", "fl": "timestamp,statuscode",
          "from": "2005", "to": "2005"}
response = session.get(cdx_url, params=params, timeout=120)
response.raise_for_status()
data = response.json()

fb2005 = pd.DataFrame(data[1:], columns=data[0])
fb2005["month"] = fb2005["timestamp"].str[:6]
print(pd.crosstab(fb2005["month"], fb2005["statuscode"]))
```

⏱️ Microsprint (5 minutes, alone or in pairs)

A status-code version of Additional Exercise 4, on a page the chapter's EDGI study watched.

Do: drop the filter for `epa.gov/climatechange`, the EPA's climate change page, from 2016 to 2018, and cross-tabulate year by status code. **Share:** the year its captures stop being 200s, and the status codes that replaced them.

```
# Microsprint: the same unfiltered query for url="epa.gov/climatechange", from "2016" to "2018";
#   cross-tabulate the year (the timestamp's first 4 characters) by statuscode
params = {"url": "epa.gov/climatechange", "output": "json", "fl": "timestamp,statuscode",
          "from": "2016", "to": "2018"}
response = session.get(cdx_url, params=params, timeout=120)
response.raise_for_status()
data = response.json()

epa = pd.DataFrame(data[1:], columns=data[0])
print(pd.crosstab(epa["timestamp"].str[:4], epa["statuscode"]))
```

💬 Discuss

- The Archive recorded each refusal accurately. For which research question are the 403 rows noise, and for which are they the finding?
- A soft 404 is a ”Page not found” page sent with a 200. Would Step 4's filter keep it or drop it, and how would you catch one?
- In week 3's dead-endpoint forensics, the way an endpoint failed was the finding. What does the status column tell us here that the pages themselves can't?

Step 7 — One version's text (10 minutes)

To measure what changed, we need the words, not the sizes. The chapter's `get_wayback_content(url, date, headers=None)` works like `get_snapshot_stats()`, but takes a date, keeps the capture's own timestamp, and returns the visible text, from BeautifulSoup's `get_text(separator=" ", strip=True)`, with its word count. It returns `None` instead of crashing when there's no capture or a request fails. Try it on one date, the start of 2010.

```
# Define get_wayback_content(url, date, headers=None), as in the chapter: format the date with
 ↪  strftime("%Y%m%d"),
#   find the closest capture and fetch it with "id_" as in Step 3, and return a dict with the
 ↪  capture's "timestamp",
#   its id_ "url", the "word_count", and the full text as "content"; return None if there is no
 ↪  capture or a request fails
# Call it for fb_tos_url = "facebook.com/terms" at pd.Timestamp("2010-01-01"); print the
 ↪  timestamp, the word count, and the first 300 characters
fb_tos_url = "facebook.com/terms"

def get_wayback_content(url, date, headers=None):
```

```
    """Retrieve a page from the Wayback Machine closest to a given date.

    Parameters
    ----------
    url : str
        The original URL (without http://)
    date : datetime
        The target date for the snapshot
    headers : dict, optional
        HTTP headers for the request

    Returns
    -------
    dict or None
        Dictionary with 'timestamp', 'url', 'content', and 'word_count',
        or None if no snapshot exists or the request failed
    """
    timestamp = date.strftime("%Y%m%d")

    try:
        api_response = session.get(
            "https://archive.org/wayback/available",
            params={"url": url, "timestamp": timestamp},
            headers=headers,
            timeout=60,
        )
        api_response.raise_for_status()
        api_data = api_response.json()

        if not api_data.get("archived_snapshots"):
            return None

        # Fetch the original capture (id_) so the extracted text doesn't include the Wayback
        ↪  toolbar and banner
        closest = api_data["archived_snapshots"]["closest"]
        snapshot_url = closest["url"].replace(
            closest["timestamp"], closest["timestamp"] + "id_"
        )
        page_response = session.get(snapshot_url, headers=headers, timeout=60)
        page_response.raise_for_status()
    except requests.exceptions.RequestException as e:
        print(f"Request failed for {url} near {timestamp}: {e}")
        return None

    soup = BeautifulSoup(page_response.text, "html.parser")

    # Extract text content
    text = soup.get_text(separator=" ", strip=True)

    return {
        "timestamp": closest["timestamp"],
        "url": snapshot_url,
        "word_count": len(text.split()),
        "content": text  # Keep the full text; the similarity analysis compares whole documents
    }
```

```
version = get_wayback_content(fb_tos_url, pd.Timestamp("2010-01-01"), headers=HEADERS)
print(version["timestamp"], version["word_count"], "words")
print(version["content"][:300])
```

⏱️ Microsprint (5 minutes, alone or in pairs)

A short version of Additional Exercise 1, which tracks a policy document of your choice.

Do: pick another policy page (another platform's terms, such as `twitter.com/tos`, or a university's privacy policy) and run `get_wayback_content()` on it at the start of 2012 and of 2022. **Share:** the page, the dates of the two captures you got, and their word counts.

```
# Microsprint: run get_wayback_content() for another policy page at 2012-01-01 and 2022-01-01,
#   sleeping 1 second between; print each capture's timestamp and word count
for day in ["2012-01-01", "2022-01-01"]:
    other = get_wayback_content("twitter.com/tos", pd.Timestamp(day), headers=HEADERS)
    if other:
        print(other["timestamp"], other["word_count"], "words")
    time.sleep(1)
```

💬 Discuss

- The function returns `None` both when no capture exists and when a request fails. When would you need to tell those apart?
- `get_text()` keeps every word on the page, menus and footers included. What will that do to the comparisons in Steps 9 and 10?
- Week 2 called terms of service ”quasi-law”. Who besides researchers would want every past version of one, and why?

Step 8 — Collect the history, with a cache (8 minutes)

Now the loop. The chapter asks for every quarter from 2005 to 2024: 77 dates and more than 150 requests, which from this room would take longer than the lab and strain the Archive. We take the start of every other year from 2006 to 2024, ten dates. Run the loop through the chapter's `get_wayback_content_cached()`, which saves each result as a JSON file in `wayback_cache/` and reads it from there next time, so that rerunning the loop on Friday asks the Archive for nothing. Sleep 1 second between dates, as the chapter does. Then save the whole collection with `json.dump()`, and load it back, as the chapter's ”Serializing Data” does.

```
# Define get_wayback_content_cached(url, date, headers=None), as in the chapter: make
 ↪  "wayback_cache"; name the file
#   by date, "wayback_cache/YYYYMMDD.json"; return its contents if it exists, or else call
 ↪  get_wayback_content() and save the result
# Loop over pd.date_range("2006-01-01", "2024-01-01", freq="2YS"), keeping each result that isn't
 ↪  None; sleep 1 second each time
# Save results to "fb_tos_history.json" (json.dump, indent=2), load it back, and print each
 ↪  version's timestamp and word count
os.makedirs("wayback_cache", exist_ok=True)

def get_wayback_content_cached(url, date, headers=None):
    """Fetch a snapshot, reading from and writing to a local cache."""
    # Key the cache file by the snapshot's target date
    cache_path = f"wayback_cache/{date.strftime('%Y%m%d')}.json"

    if os.path.exists(cache_path):  # Already fetched — skip the network
        with open(cache_path, "r") as f:
            return json.load(f)
```

```
    result = get_wayback_content(url, date, headers=headers)
    if result is not None:  # Save for next time
        with open(cache_path, "w") as f:
            json.dump(result, f)
    return result

dates = pd.date_range(start="2006-01-01", end="2024-01-01", freq="2YS")
results = []
for date in dates:
    result = get_wayback_content_cached(fb_tos_url, date, headers=HEADERS)
    if result is not None:
        results.append(result)
    time.sleep(1)  # Rest between iterations — the Archive is a nonprofit
print(f"Retrieved {len(results)} snapshots from {len(dates)} time points")

with open("fb_tos_history.json", "w") as f:
    json.dump(results, f, indent=2)
with open("fb_tos_history.json", "r") as f:
    results = json.load(f)
for r in results:
    print(r["timestamp"], r["word_count"], "words")
```

💬 **Discuss**

- The cache names each file by date alone. What happens if you call `get_wayback_content_cached()` for `twitter.com/tos` on a date you've already fetched for Facebook?
- Did two of our dates land on the same capture? What would that do to Step 10's change timeline?
- Week 1 saved pageviews as CSV. Our results are week 4's list of dictionaries, each with a long text. Why does JSON suit them better?

Step 9 — Bag of words: the first version and the last (6 minutes)

A bag of words represents a document by how often each word appears in it, ignoring their order. First fetch the book's list of 198 stopwords, words like the and of that are too common to tell versions apart, from `https://raw.githubusercontent.com/cuinfoscience/Web-Data-Science-Book/main/data/stopwords-en.txt`. Then define the chapter's `preprocess_document()`: gensim's `simple_preprocess()` splits the text into lowercase tokens, and we drop the stopwords. A gensim `Dictionary` numbers the words, `doc2bow()` turns each document into a bag, and `SparseMatrixSimilarity` gives their cosine similarity: 1 for the same words in the same proportions, 0 for no words in common.

```
# Fetch the stopword list (the address above) with the session and make it a set; print how many
 ↪  there are
# Define preprocess_document(text): simple_preprocess() the text, then drop the stopwords
# Preprocess the first and last versions in results; build a Dictionary, a doc2bow corpus, and a
#   SparseMatrixSimilarity index from them, and print the similarity of the two versions
from gensim.utils import simple_preprocess
from gensim import corpora, similarities

STOPWORDS_URL =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/Web-Data-Science-Book/main/data/stopwords-en.txt"
response = session.get(STOPWORDS_URL, timeout=30)
response.raise_for_status()
stop_words = set(response.text.split())  # split() breaks the text at every line break
print(f"{len(stop_words)} stopwords, such as {sorted(stop_words)[:5]}")
```

```
def preprocess_document(text):
    """Tokenize, lowercase, and remove stopwords from a document."""
    tokens = simple_preprocess(text)  # Tokenize and lowercase
    return [token for token in tokens if token not in stop_words]

doc_v1 = preprocess_document(results[0]["content"])
doc_v2 = preprocess_document(results[-1]["content"])
print(f"Version 1 ({results[0]['timestamp'][:8]}) tokens: {len(doc_v1)}")
print(f"Version 2 ({results[-1]['timestamp'][:8]}) tokens: {len(doc_v2)}")

documents = [doc_v1, doc_v2]
dictionary = corpora.Dictionary(documents)
corpus = [dictionary.doc2bow(doc) for doc in documents]
index = similarities.SparseMatrixSimilarity(corpus, num_features=len(dictionary))
sims = index[corpus[0]]
print(f"Similarity between versions: {sims[1]:.3f}")
```

💬 Discuss

- not, no, and nor are on the stopword list. Think of a sentence in a terms of service whose meaning flips when not is added. Would our score notice?
- Menus, footers, and cookie notices appear in both versions. Do they push the score toward 1 or toward 0?
- Week 6's parsers kept one table or one infobox; this one keeps every word on the page. When is each the right choice?

Step 10 — A change timeline (8 minutes)

Compare each version with the one before it, and call one minus their similarity the change score: 0 for no change, 1 for nothing in common. Build one `Dictionary` and one index for all the versions, as the chapter's "Visualizing Document Change Over Time" does. The chapter plots the scores against the transition's number; we plot each against the date of the later capture, so a spike can be matched with an event.

```
# Keep the results that have content, preprocess each, and build one Dictionary, corpus, and
 ↪  SparseMatrixSimilarity index
# For each version after the first, compute 1 minus its similarity to the version before
 ↪  (index[corpus[i-1]][i])
# Plot the change scores against the later capture's date, with markers, a title, and labeled
 ↪  axes; print them
kept = [r for r in results if r and r.get("content")]
processed_docs = [preprocess_document(r["content"]) for r in kept]

dictionary = corpora.Dictionary(processed_docs)
corpus = [dictionary.doc2bow(doc) for doc in processed_docs]
index = similarities.SparseMatrixSimilarity(corpus, num_features=len(dictionary))

change_scores = []
for i in range(1, len(corpus)):
    sim = index[corpus[i-1]][i]
    change_scores.append(1 - sim)  # Convert similarity to change score

when = pd.to_datetime([r["timestamp"] for r in kept[1:]], format="%Y%m%d%H%M%S")
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(when, change_scores, marker="o")
ax.set_xlabel("Date of the later capture")
ax.set_ylabel("Change score (0 = identical)")
```

```
ax.set_title("facebook.com/terms: change from the version before")
plt.tight_layout()
plt.show()

for day, score in zip(when, change_scores):
    print(day.date(), f"{score:.3f}")
```

⏱️ Microsprint (6 minutes, alone or in pairs)

A short version of Additional Exercise 3, which compares every version with every other.

Do: compute the similarity of every version with every other (`index[corpus]` is the whole matrix) and draw it as a heatmap with `plt.imshow()`, both axes labeled with the capture years. **Share:** the pair of years whose versions differ most, and any run of years where the page stayed the same.

```
# Microsprint: matrix = index[corpus]; draw it with plt.imshow(), label both axes with the
 ↪  capture years,
#   and add a colorbar and a title; print the least similar pair of years
matrix = index[corpus]
years = [r["timestamp"][:4] for r in kept]

fig, ax = plt.subplots(figsize=(6, 5))
image = ax.imshow(matrix, cmap="viridis")
ax.set_xticks(range(len(years)), years, rotation=90)
ax.set_yticks(range(len(years)), years)
ax.set_xlabel("Year of capture")
ax.set_ylabel("Year of capture")
ax.set_title("facebook.com/terms: similarity of every pair of versions")
fig.colorbar(image, label="Cosine similarity")
plt.tight_layout()
plt.show()

i, j = divmod(matrix.argmin(), len(years))
print(f"Least similar: {years[i]} and {years[j]} ({matrix[i, j]:.3f})")
```

💬 Discuss

- Pick the biggest spike. What happened at Facebook around then, and how would you check it with archived sources rather than from memory?
- Captures two years apart can't see a change that came and went between them. What would the chapter's quarterly dates add, and what would they cost the Archive?
- The chapter's graduate exercise warns that observed change is confounded with attention. Where could that confound enter this plot?

Step 11 — What did we learn? (5 minutes)

In a few sentences, as comments or in a Markdown cell:

- How many distinct versions of `facebook.com/terms` did the CDX Server list in Step 4, and how many different captures did our ten dates reach in Step 8? What does the gap mean for Step 10's timeline?
- In Step 6, what share of `facebook.com`'s captures in 2005 weren't 200s? What would a study that kept only the 200s have missed?
- Which transition changed most in Step 10? Does it show up as a jump in Step 5's sizes?
- What would you write in a methods section about how the Wayback Machine shaped these results?

Recommended Exercises ahead (in the chapter's companion notebook, due Sunday):

- Step 1, query the CDX Server for your page's history, and handle a 429: Step 4's query, and Step 0's retrying session.
- Step 2, build the capture DataFrame and print its date range and size: Step 4.
- Step 3, plot captures per year and page size over time: Step 5's `resample()` and two-panel plot.
- Step 4, fetch the earliest and latest captures with `id_` and cache them: Step 2's `id_`, and Step 8's cache.
- Step 5, extract the text of both captures and measure their similarity: Step 7's `get_text()`, and Step 9.
- Step 6, interpret: Steps 5, 6, and 10, and the Discuss questions about gaps and capture bias.

```
# Your answers here, as comments
# To write once the key has run: answer each question with the numbers its outputs show.
```

Friday: from the lab to the textbook (15 minutes)

What did the lab turn up that the chapter should know? Some places to look:

- Step 3's table, next to the chapter's illustrative one in ”Comparing Multiple Snapshots”. Should the chapter print real output, with its date?
- Step 8's cache, which names files by date alone (the Discuss question). Is that a bug in the chapter's code, and what would you name the files instead?
- The chapter's collection loop, which calls `get_wayback_content()` without `headers`, so its requests go out with no User-Agent of ours, and with no `timeout`.
- Step 5's `length`, which the chapter calls ”page size”. Is ”compressed record size” clearer, or just more words?
- A cell that waited, retried, or failed today, and what the chapter's ”Common Issues to Debug” should say about it.
- A microsprint that would make a good exercise.

`handouts/common/revision-framework.md` says how to propose a revision.